# DPH Engineering Guide

*Delphi Project Handoff*

Windows desktop experimental software

Last changed 12 October 2026

For a Delphi developer building maintaining and validating DPH

This guide describes the implemented application and its evidence limits. Use the phase acceptance records for exact measured results and unresolved release gates.

# Contents

Architecture and boundaries....4
Rework and acceptance status....4
Build and deployment....4
Input acquisition and project scope....5
Syntax contracts and limits....5
Symbols and typed relationships....5
Control flow and impact....6
SQLite generations and durability....6
Evidence and immutable history....6
Background coordination and cached views....7
FMX navigation and visualization....7
Curated knowledge and review....7
Canonical publication and privacy....8
HTML guides and bundle validation....8
Translation boundary....9
Verification and release discipline....9
Current semantic cache and extended evidence....9
Current schema and authored document contracts....10
Native review and learning projections....10
Saved observation persistence....10
Regression commands and evidence....11
Rebuild and inspect these manuals....11
Business glossary implementation and checks....12
Offline translation and localized publication....12
Native validation and release evidence....13
Structured project profile contract....13
Quality analysis and revision comparison....14
Keyboard inspector implementation and validation....14

Command inventory implementation and validation....15
Dynamic control relationships and validation....15
Layout rule explanations and validation....15
Component dependencies and attributed metadata....16
Resolve form inheritance and frame overlays....17
Store attributed action purpose mappings....17
Compare attributed screen evidence and captured layouts....18
Build a safe captured database connection inventory....18
Derive captured database engine and version evidence....19
Inspect captured database schema evidence....20
Extract captured column declarations and constraint text....20
Resolve captured key membership and references....21
Capture index structures and SQL reference evidence....22
Capture stored-object definitions and dependencies....23
Project captured entity relationship metadata....24
Maintain attributed schema dictionaries....25
Resolve captured dataset and field bindings....26
Recognize captured database operation requests....27
Trace captured transaction boundaries....27
Capture field validation and transformation evidence....28
Connect source evidence into scoped lineage....28
Project captured migration and version evidence....29
Project captured file formats and contracts....30
Maintain attributed target retention requirements....30
Maintain target data quality evidence and report imports....31
Maintain target backup and restoration evidence....32
Maintain structured business process records....32
Maintain ordered departmental workflow evidence....33
Maintain end-to-end process projections....33
Maintain the business rule catalog....34
Source and revision evidence....34

## Architecture and boundaries

DPH is an independent Delphi FMX application. It has no converter runtime or build dependency. The pinned VCL2FMXConverter v6.0.0 corpus is test input only, at commit e1fd10b2d3b06f2fc3f726b8eb7f234a54faf63d. No converter source was copied into the current analyzer. Preserve the original planning documents and global agents.md byte for byte.

The pipeline is selected project, bounded inventory, read-only byte acquisition, lexical and conditional processing, Pascal and form IR, semantic relationships and control flow, revision-consistent SQLite capture, cached workspace projection, curated knowledge and derived publication. Contract data remains separate from FMX. Every unsupported construct or unavailable dependency must remain diagnosable rather than being promoted into false completeness.

Application controls are serialized in DPH.UI.Main.fmx, DPH.UI.ProjectProfile.fmx and DPH.UI.Glossary.fmx and editable in the IDE. List contents and paint-box geometry are data projections. No scanned component class is instantiated. Avoid adding runtime-only controls, Pascal class helpers or an alternative authoritative store without explicit authorization.

## Rework and acceptance status

The owner rejected the earlier WP7 delivery as inadequate for learning and handoff. The statement that implementation through WP7 was complete has been withdrawn. docs/Rework_Requirements_Audit.txt compares the original plan with the implemented product and outstanding work. Historical WP reports remain historical evidence, not acceptance of the reworked experience.

The rework adds a first-launch Getting started course, deterministic explanation engine, guided learning workspace, source-comment context, event traces, labeled and collapsed procedure-flow views, attributed screenshot imports and richer project-specific publication. Run tools/build-learning.cmd with an optional pinned reference database path for dedicated learning and real-WebView2 tests. The source-based first-time-programmer exercise still needs a human participant.

The implementation tracker records expanded semantic reuse, resource interpretation and retention/continuity controls. Their documented bounds still apply. Source safety, privacy, attribution and exact generation links are retained. Do not turn an automated pass count into a claim that every requirement or the human onboarding goal is finished.

## Build and deployment

The installed compiler is RAD Studio 37.0.59082.6021. Load C:\Program Files (x86)\Embarcadero\Studio\37.0\bin\rsvars.bat before compiling. tools/build-wp0.cmd builds DelphiProjectHandoff.dproj with elevated MSBuild and runs the FMX resource smoke test. Output is bin/Win32/Debug, with DCUs in dcu/Win32/Debug. The initial executable target is Windows Win32.

Deploy the executable, all nine immutable migration files, and the architecture-matched WebView2Loader.dll. The reviewed Microsoft WebView2 SDK package is pinned to 1.0.4258.31; retain tools/WebView2/LICENSE.txt with binary redistribution. The browser loads the DLL by its absolute executable-folder path and activates Edge only when preview is requested. There is no intended IE fallback.

The repository standard folders include bin, dcu, docs/guides, docs/pdf, export, help, images and icons, samples, source distributions and tools. Deploy help/index.html for first-launch education. Reference source and generated test databases remain ignored. Use the explicitly approved Python 3.13 executable for project tooling and the installed LibreOffice, never Microsoft Word.

## Input acquisition and project scope

DPH.Project.Inventory follows the selected project, explicit references, uses clauses, include paths and approved external roots. Project XML is passive: targets and imported build scripts are not executed. Canonical paths are project relative and normalized; traversal, ambiguous macros, reparse paths, cycles and oversized acquisition are rejected or diagnosed. A repository-wide count of Pascal files is not the build scope.

DPH.Project.Source opens input read only, checks the final opened Windows handle path, captures bytes, records SHA256 and verifies decoding by round trip. Unicode byte offsets, line endings and explicit legacy encodings remain in the evidence contract. The maximum acquired source is 16 MiB. Binary forms are parsed as data; unsupported resources remain gaps.

A source viewer must re-acquire the live file and match the captured hash before using a cached range. Externally referenced sources use their explicit canonical inventory identity. Never invent a neighboring source path from a cached filename or execute input to fill an analysis gap.

## Syntax contracts and limits

DPH.Parse.Model defines IR schema 1, analyzer DPH-WP2-6, token and node limits, source ranges and diagnostics. Lexing preserves original tokens and conditional information. Preprocessing evaluates supported expressions, explicit defines and includes while retaining uncertainty for unknown branches. An unavailable compiler symbol is not automatically a false condition.

The Pascal parser extracts declarations, routines, anonymous methods and supported statements into IR. Declaration and implementation occurrences remain distinguishable before semantic unification. The form parser accepts supported text and binary DFM/FMX structures without loading component packages. The inventory reads relevant project configuration and resources through bounded adapters.

Current bounds include 250000 tokens, 100000 IR nodes, nesting 128 and 10000 diagnostics. Opaque assembler, malformed syntax and unsupported grammar are partial. Tests use exact golden IR, encoding edge cases, cyclic includes, passive malicious imports and changed source. Maintain source byte ranges and diagnostics when extending support.

## Symbols and typed relationships

DPH.Analysis.Model defines semantic schema 1 and analyzer DPH-WP3-15. Stable symbol keys contain qualified identity, signature and conditional variant. DPH.Analysis.Symbols unifies supported routine prototypes and bodies, resolves cross-unit declarations, typed receivers and suitable overloads, associates form events and records anonymous callbacks.

Relationship certainty is confirmed_static, possible_dynamic, inferred, runtime_observed or unresolved. Passive scanning never claims runtime_observed. A uses dependency is not a call. Known lexical calls, serialized bindings and declared types carry derivation rules and source ranges. Recognized asynchronous API spelling can establish a possible scheduling relationship without proving execution order.

Method pointers, unresolved receivers, dynamic dispatch, generated types and framework behavior must retain uncertainty. Basic reads, writes, callback assignment, static SQL spelling and report-model rendering are conservative analyses. Renames are distinct identities until continuity is reviewed; offsets alone must not reassign knowledge.

## Control flow and impact

DPH.Analysis.Flow builds per-routine control-flow nodes and typed edges for supported sequences, branches, loops, returns and exception/finally paths. Routine identity, source hash, range, node kind and expression remain available to inspectors and exports. Flow is bounded to 100000 nodes, 300000 edges and nesting 128.

Impact traversal is possible reachability through supported relationships. Do not label it proved runtime impact. The UI bounds a displayed graph to 120 nodes and 960 edges, while the stored semantic graph is the fuller evidence model. Uncertain edges remain visually distinct. Unsupported flow does not become a fabricated successful path.

## SQLite generations and durability

TDPHKnowledgeStore owns one FireDAC SQLite connection on its creating thread and serializes writer access through a named mutex. Foreign keys and synchronous FULL are enabled. Default journal mode is DELETE; WAL is an explicit local-store option. Direct cross-thread writes are rejected. SQLite 3.42.0 with FTS5 was measured by the storage tests.

Schema 9 is created through nine checksum-verified migrations. Existing migration text is immutable. A newer incompatible schema is rejected safely. Failed upgrade DDL and ledger changes roll back, and an upgrade from an existing schema makes a recoverable pre-migration database backup. Interrupted running generations recover as failed, preserving the previous published head.

ObserveManifest records current fingerprints independently of the capture. BeginGeneration freezes configuration and manifest identity. Syntax and semantic facts are staged in transactions. Before publication, inputs and configuration are revalidated. Only a coherent complete generation becomes the current head. Partial, failed or superseded captures cannot clear the exact-input stale state or overwrite a prior current head.

## Evidence and immutable history

Normalized tables include projects, configurations, source files, file revisions, generations, generation files, entities, entity versions, evidence, relationships, component properties, control-flow nodes and edges, knowledge records and revisions, links, review tasks, translations, artifacts, export jobs and audit events. Foreign keys tie evidence and relationships to their producing generation.

Triggers prevent mutation of terminal generations, source revisions, captured facts, configurations and authored knowledge revisions. Evidence ranges must fit the captured source bytes and use the generation analyzer version. Historical file and entity identities cannot be changed casually. A missing source is retained as missing; its historical annotations survive.

Modification detected, analysis stale, knowledge review needed, analysis failed and reassessment requested are separate persistent flags. A UI summary is a projection of these states, not a replacement for them. Parser success must not clear human review.

## Background coordination and cached views

DPH.Workspace.Worker is a persistent TThread with bounded request and response queues. It initializes COM for passive MSXML parsing, owns database operations in the worker, and retains analyzer instances across requests. FMX polls responses with a timer, avoiding queued closures that outlive the form.

A stopped Delphi queue can report success while returning an empty item. Poll checks both the wait result and a non-nil response; shutdown tests guard against a destructor loop. Cancellation is checked between stages and marks a staged running generation failed when appropriate. It is not a promise of interruption inside every parser loop.

Workspace projections are managed JSON files in database.dphdb.views named by SHA256 and registered in artifacts. Cached opening verifies the managed path, size, checksum and generation. Cached facts are delivered before automatic verification when the original source entry is available; unavailable source remains detached. Connected checks compare manifest fingerprints and selection identity every ten seconds. Syntax and routine semantic caches are bounded and binding-context aware; conservative postprocessing remains separate from reused routine facts.

## FMX navigation and visualization

DPH.UI.Main binds its published controls to the FMX resource. Explorer queries are paged at 250 rows; data filtering supports kind, text and qualified scope. History and bookmarks use canonical keys, with surviving same-project bookmarks retained across reassessment. Inspector output retains source metadata and serialized properties. Source memo navigation is permitted only after live hash validation.

OpenSelection queues a candidate without replacing the current database, source or workspace. PollTimer installs it only after the worker returns accepted captured evidence. The worker preflights source identity before creating a database; cached opens check captured XML ProjectGuid and Pascal program declarations before returning a view. Rejections retain the current workspace, and rejected startup caches show Getting started. Inventory identity also covers referenced project groups. DatabaseDialog.InitialDir is the design-time relative default; startup resolves it under the deployment root, creates that folder, and applies the same path to OpenCacheDialog. Distribution packaging copies only the Databases instructions, never its databases or views.

Graph paint boxes draw bounded evidence data with palette and geometry values supplied by design-time style rectangles. Screen reconstruction draws approximate source-derived rectangles without creating components. Its gallery is bounded to 60 forms. Runtime styles, custom painting and inherited resources are not silently fabricated.

Keep lifecycle and ownership explicit: the view owns its JSON document, UI arrays own their records, responses transfer document or knowledge ownership to the main form, and the worker frees unconsumed responses during shutdown. Guard unsaved authored text before changing projects or replacing editor state.

## Curated knowledge and review

SaveKnowledge requires title, body, a named author, a supported kind and a captured generation. Existing records require their expected revision and stable kind. Ordinary titles stay stable; structured glossary names can change while their identity and history remain intact. Each edit appends a needs_review revision, preserves links and opens a pending task. Knowledge bodies are bounded to 1 MiB. Authored text is never regenerated from parser output.

ReviewKnowledge requires the expected current revision, a named reviewer and a matching captured source/configuration. The connected worker observes current source before accepting a review. A missing linked entity cannot be verified under a replacement name. Verification, rejection or retirement appends a revision and records attribution; history remains append only.

Manifest and configuration changes conservatively invalidate active knowledge, derived artifacts and translations. A partial static capture may support review of an attributable statement against available evidence, but its partial coverage remains visible. Local names are not authenticated enterprise identities. Human verification and automatic static derivation remain different evidence categories.

## Canonical publication and privacy

DPH.Publish.Model constructs one versioned documentation model from a captured workspace and current curated knowledge. It carries generation, configuration, input/context fingerprints, detached/stale state, symbols, typed relationships, control flow, source references, human history, coverage and explicit unknowns. Full and task-scoped views share those facts.

Task scope selects the requested identities and complete direct incident context. It rejects missing or excluded seeds and rejects an oversized task rather than silently truncating a full export. External source roots and sensitive filename categories are excluded by default. Explicit relative prefix exclusions remove file-derived symbols and relationships. Source snapshots and generated AGENTS.md are absent by default.

Redaction recognizes common credential assignments, private keys, AWS access keys and JWT-shaped tokens throughout string data. It is defense in depth, not a guarantee that every project-specific secret format is recognizable. Require explicit exclusions and a disclosure review for sensitive content. No translation or AI provider is contacted by normal publishing.

## HTML guides and bundle validation

DPH.Publish.Export writes a new destination outside source. The offline HTML uses encoded imported text, local search and styles, internal anchors, print CSS and a restrictive content security policy. The default WebView2 policy permits only the authorized local index and about:blank; publication explicitly enables hash-named object pages in its managed objects folder. Help retains the index-only policy; learning explicitly permits content-hashed pages only inside its owned preview directory. There is no arbitrary native bridge. Data does not supply executable markup.

JSONL indexes contain symbols, relationships, forms/events, data usage, configuration, flow nodes, knowledge and evidence references. Manifest and checksum files accompany canonical.json. The validator checks filenames, hashes, expected input fingerprint, canonical identity, source hash/range, generation and cross-reference integrity. A stale manifest is rejected by default. Unsigned hashes establish integrity relative to the index, not author authenticity.

Editable engineering and user DOCX guides derive from the same sections and evidence model. They include title, last changed capture identity and a changeable TOC field. LibreOffice provides companion PDFs; the project rendering tool verifies cached page numbers and renders page images for layout inspection. Missing business procedures stay explicitly absent rather than being invented.

## Translation boundary

IDPHTranslationProvider exposes provider identity, version, external status and prose translation. Protected segments retain identifiers, SQL, paths, hashes and protocol keys byte for byte. Cache keys include source content, revision, source/target locales and provider/version. Changed prose produces a fresh pending entry; named review applies to the exact cached translation.

TDPHAtlasAdapter reads explicitly selected DAT JSON packs offline. The independent Apache-2.0 reader snapshot and license reside under dependencies\DelphiAppTranslation; VCL2FMX is unchanged and is not a build or runtime dependency. No network provider is configured. Tests exercise actual packs, persistent cache, terminology changes, source invalidation and exact-content named review. The offline GTranslate region is disabled, with local bilingual labels and sorting; do not activate network translation merely because a generic widget name appears in documentation.

## Verification and release discipline

Run the phase build/test scripts elevated with the installed toolchain. WP0 checks FMX resource ownership; WP1 checks storage, crash recovery and migrations; WP2 checks inventory, encoding, syntax and source safety; WP3 checks semantics and pinned reference traces; WP4 checks workspace/background behavior; WP5 checks immutable authored history and pending review; WP6 checks publication, exclusions, sanitization, hashes and translation protection.

Use exact input hashes before and after analysis, including failures. Preserve original reference and instruction files. Keep tests that demonstrate invariants rather than tests that merely mirror implementation. New source, validation failure or unresolved risk justifies another targeted test; a passing unchanged phase does not need endless repetition.

H: development backups are enabled. Allow at least 60 seconds for this USB drive to wake before assessing availability. Git remains enabled: review intended changes, commit validated work and push both public and private remotes without force or history rewriting. Remove a stale zero-byte index lock only after verifying no Git process runs. WP7 records measured performance, portability, security and explicit acceptance evidence. Independent first-time programmer acceptance and unknown historical business intent cannot be invented by automation.

In the historical pre-rework benchmark on the recorded Core i7-1165G7 host with eight logical processors and about 16 GB RAM, the pinned reference pipeline measured 82.6 seconds for full capture, 2.87 seconds for cached opening and 75.9 seconds for incremental reassessment. The message-pump maximum gap was 64 ms. Peak test-process working set was about 1.21 GiB, including repeated captures, JSON cloning and FMX/WebView2 tests. These are baseline measurements, not agreed hardware-independent budgets. Large syntax-cache fallback and conservative semantic recomputation explain the modest reassessment speedup. Formal acceptance retains independent complex-flow review, missing business/user content, AC-25 and performance-budget agreement as outstanding.

## Current semantic cache and extended evidence

The current analyzers are DPH-WP2-6 and DPH-WP3-15 with IR and semantic schema 1. Routine semantic cache reuse is bounded to 512 records and 32 MiB. Binding context hashes invalidate affected reuse. Synthetic startup and postprocessing occur after reusable routine facts, preserving their relationships to current identities. This is bounded evidence reuse; it is not a proof of an optimal incremental solver.

Supported extensions include typed aliases, inheritance and accessors, startup entry and unit phases, case/goto/with control flow, asynchronous operation handles and captures, cancellation and event waits, formal arguments and result producers, static SQL projection, output-writer lineage, lifetime and exception relationships, explicit test-call links, and qualified form navigation. Every relationship retains resolution, source range/hash and rule. Unsupported grammar, aliases and dynamic behavior remain diagnosed or qualified.

## Current schema and authored document contracts

Schema 9 uses checksum-verified migrations 001 through 009. document_section_versions associates immutable authored revisions with stable section ID, document kind, audience and source generation. knowledge_revision_links preserves ordinal, target kind/identity, captured generation, role and label. knowledge_questions preserves the exact deterministic question identity and captured JSON associated with a response record.

SaveKnowledge appends immutable text and link snapshots. Empty LinksJSON preserves the prior snapshot; an explicit JSON array replaces it. Entity, relationship and evidence targets remain tied to generation identities; screenshot targets reference managed artifacts; section targets pin an exact knowledge revision. Current reverse section references open review when their targets change. Verify rejects obsolete rich-link generations or target revisions.

Question identity hashes kind, entity key, range, source hash, prompt, rule and context fingerprint. Generation is retained separately, allowing unchanged evidence to keep its question identity across captures. An answer needs actual text after Answer. Only verified content with no pending task suppresses an exact question. Export converts question metadata to typed JSON before recursive redaction.

## Native review and learning projections

ReviewTasks provides bounded pending tasks, prior and current authored text, baseline/current configurations, affected object signatures, types, source hashes and relationship comparisons. A missing baseline is reported. Native controls invoke existing optimistic-revision and connected-source review checks; they do not create a separate approval authority.

RoutineLesson uses the routine CFG and an indexed outgoing-edge lookup. It explains source points, actual expressions and destination lines, with explicit loop, branch, early-return, cleanup and exceptional semantics. The same lesson is used by ObjectText and ObjectHTML; exported document prose and offline object pages therefore share its source interpretation. Its bounds are 60 points and eight outgoing paths per point.

Preview HTML files have immutable content-hashed names, tracked for cleanup. BrowserPolicy allows them only in the owned preview directory when explicitly enabled. This avoids rewriting a file held by WebView2. Arbitrary filenames and external navigation remain blocked. Native browser tests render the app pages; they do not execute the analyzed application.

## Saved observation persistence

ImportObservation accepts explicitly selected UTF-8 text extracts up to 256 KiB with named contributor and run context. It checks generation and optional entity existence, captures original byte SHA256 and linked file-revision SHA256, and stores content plus attribution in a managed hash-named JSON artifact of kind runtime_observation. No schema migration is needed for this artifact kind.

ObservationsJSON verifies managed identity, size, checksum and generation before returning the newest 50 records. The response owns the array until PollTimer transfers it to the form. LoadView clears old observation rows; historical records cannot navigate as current. Saved text stays in a native read-only memo rather than an executable browser surface.

ManagedArtifactPaths includes observations for explicit portable backup. Retention protects their associated captured views. Publication currently omits observations rather than automatically disclose potentially sensitive logs. Contributor claims do not set runtime_observed on static analysis or create an automated causal trace.

## Regression commands and evidence

Run commands from the project root in an elevated Command Prompt so RAD Studio can read its environment. tools\build-wp0.cmd builds the app and checks FMX bindings. tools\validate-completion-wp1.cmd serially builds app, catalog, lifecycle, workspace, knowledge, selection and startup tests, then removes recognized disposable fixture directories.

Run tools\build-wp1.cmd for 46 storage checks, tools\build-wp2.cmd no-report for discovery/resource checks, and tools\build-wp3.cmd to compile semantic tests. Then explicitly run bin\Win32\Debug\DPH.Tests.Semantics.exe "C:\New Delphi Projects\Delphi Project Handoff"; the build script alone does not run semantic assertions.

tools\build-learning.cmd exercises managed images, cached supplemental-evidence recovery, actual WebView2 rendering, source walkthroughs and collapsed CFGs. --no-ui isolates deterministic checks and is not native acceptance. tools\build-wp6.cmd checks canonical scope, redaction, link integrity, publication and translation protection. Use exact log files and current input hashes for claims; historical reports are not automatically refreshed acceptance.

A native-learning test once dereferenced an absent image array on a later background-verification response. It now checks the initial cached document/image response directly. Source attachment preserves supplemental-image warnings. Inspection disproved a proposed constructor race, and no worker thread-start change remains.

## Rebuild and inspect these manuals

tools\build-guides.py generates the two application manuals from maintained operating content and current implementation hashes. Use the approved Python executable C:\Users\marti\AppData\Local\Microsoft\WindowsApps\python.exe. tools\render-guides.ps1 renders with LibreOffice, populates cached page numbers in editable TOC fields, renders again and writes companion PDFs under docs\pdf.

Inspect every final page PNG under bin\guide-qa for clipping, broken headings, unreadable code and TOC page numbers before delivering a guide. Rendered PNGs are QA outputs, not a mock application UI. Actual included interface images are identified by their capture source and fixture; they do not establish runtime behavior of a selected project.

Regenerate manuals after later translation, handoff-contract or source-navigation changes. The source evidence appendix records inspected file hashes, while docs\guide-source-validation.json provides a machine-readable manifest. A successful generator run alone is not proof that the writing, current behavior or layout is correct.

## Business glossary implementation and checks

DPH.Knowledge.Glossary validates a business_glossary structured record with term, aliases, definition, examples and relevance/evidence context. Names normalize case and repeated whitespace. Terms/aliases are bounded to 128 characters, 32 aliases per term, definitions/examples to 16384 characters, context to 4096 characters and the record to 64 KiB. Unknown fields, duplicate fields and non-text aliases are rejected.

The existing migration 009 structured_json column stores the exact term fields on each immutable knowledge revision; no new migration is needed. SaveKnowledge enforces glossary kind, user document and all-reader audience, generated body agreement, project-local name uniqueness, type preservation and optimistic revision checks. Glossary titles can change while the knowledge ID and prior structured revisions remain intact. Retired records continue reserving their current names.

DPH.UI.Glossary.fmx defines every term editor control for the IDE/Object Inspector. DPH.UI.Main.fmx defines the glossary list, filter, detail memo and commands. Apply prepares a draft in the existing Knowledge pipeline. The modal editor retains a cloned saved revision so a background refresh cannot invalidate its input pointer. Current source links, explicit review, search and stale-source tracking use existing repository contracts.

DPH-LEARNING-8 publication emits a dedicated HTML glossary index and indexes/glossary.jsonl from the canonical knowledge records. Both user and engineering DOCX use those same records. Validate compares the glossary JSONL with the canonical glossary subset; recomputing the artifact hash cannot conceal divergent content. Older publication versions remain readable under their prior contract.

From an elevated Command Prompt run cd /d "C:\New Delphi Projects\Delphi Project Handoff", then tools\build-glossary.cmd. The suite uses the fixed ignored bin\glossary-validation directory and controlled fixtures. It checks field validation, normalized conflicts, persistence, rename, immutable revisions, stale edits/reviews, isolation, source-linked invalidation, search, publication agreement/exclusions/tampering and IDE-designed native form workflows at normal/minimum sizes. Native images are FMX PaintTo renders of actual instantiated controls; they are not screenshots of the analyzed target application.

Build the application with tools\build-wp0.cmd. Run tools\build-wp5.cmd for the existing knowledge/link/review regressions and tools\build-wp6.cmd for publishing/translation regressions. A FAIL or nonzero exit means the corresponding check failed. See docs\phase1-glossary-validation.json and the glossary completion note for actual current run evidence.

## Offline translation and localized publication

Migration 008 adds the persistent translation cache key and JSON record. Cache identity covers saved knowledge revision, requested locales, provider pack hash, source segments and terminology. SQL state overlays cached metadata so source invalidation remains authoritative after database reopen. Approval requires current source review, complete translated prose and a named reviewer; it never updates authoritative source text.

TDPHAtlasAdapter uses DAT.Runtime.LanguagePack plus its two copied core dependencies, original license and hashed snapshot provenance. Selected packs are bounded to 4 MiB, strictly UTF-8 and application scoped. Exact source phrase lookup and base-language fallback are explicit. Common code, SQL, paths and identifiers are protected during segmentation; unusual unmarked identifiers still need human review.

DPH-LEARNING-7 publication writes localization.json and, when a locale is requested, a hashed HTML page and Localized_Sections.docx. Verified exact-revision cache entries are eligible only while the canonical source section is reviewed and has no pending tasks. Redacted source bodies use source fallback. The supplementary document explicitly limits its scope to authored bodies.

Source navigation is separately opt-in. Only scoped captured text with matching source hashes is rendered as encoded, redacted HTML. Task masks use captured noncontainer ranges and original line starts. source-navigation.json records included line numbers and omission reasons. Validate checks scope, file identity, line order/counts, generations and disclosure metadata. No original source snapshot is shipped.

## Native validation and release evidence

The design-time Validate bundle controls dispatch validate_bundle to the worker before a database connection is required. The owner supplies an expected fingerprint independently. Stale rejection is default and historical inspection is an explicit qualification. Native reports distinguish integrity from human approval.

The Win32 project enables DCC_LargeAddressAware in its IDE project options. The hardening and learning harnesses use the equivalent PE flag. The expanded pinned corpus exposed an out-of-memory failure after repeated captures and native workspace loading under the former address limit. Keep memory measurements and actual large-corpus validation with this change; the linker flag does not make memory consumption unbounded or establish a release budget.

Run tools\validate-wp7.ps1 elevated for the pinned reference hardening suite and before/after hash proof. It reads the current schema constant rather than an obsolete hardcoded version. The suite measures full capture, cached open, reassessment, message-pump gaps and peak process working set on recorded hardware. Measurements require owner-agreed budgets before release acceptance.

The independent first-time programmer and source-review worksheet is docs\WP7_Maintenance_Exercise.txt. Preserve actual participant answers, source citations, uncertainty judgments, assistance, timings and defects. Do not substitute automated tests or prepared instructions for AC-25 or independent explanation review.

## Structured project profile contract

TDPHWorkspaceView.ProjectMembers projects captured syntax files into a selected-scope catalog without filesystem discovery. DPH.UI.Main.fmx defines separate Known databases and Selected project subpages, with a native list and evidence memo. Program, library and package declarations retain source references; project properties and references retain captured values and child conditions. Parent conditions may apply even when a child has none recorded. tests\DPH.Tests.ProjectCatalog.dpr checks a selected group, missing referenced project, output kinds, settings, native selection and input immutability.

Migration 009 adds an object-valued structured_json field to append-only knowledge revisions. TDPHProjectProfile validates 15 known text fields, required purpose and attribution, per-field bounds and a 64 KiB record bound. SaveKnowledge enforces stable project scope, title, kind and generated body, prevents ordinary-record conversion, duplicate profiles and loss of typed data, and preserves structured fields during named review and continuity revisions. Publication decodes typed metadata before recursive redaction, including revision history.

DPH.UI.ProjectProfile.fmx defines every editor control at design time. Apply returns an attributed draft to the existing Knowledge save pipeline. It does not bypass unsaved-edit guards or automatically approve

information. tests\DPH.Tests.Phase1.dpr verifies persistence, review, stale editors, history, field bounds and native form behavior; publishing regressions check canonical, HTML and DOCX inclusion.

## Quality analysis and revision comparison

DPH.Analysis.Quality emits DPH-QUALITY-1 evidence during workspace capture. Routine metrics, eligible token-body equality, conservative missing-inbound and disconnected-CFG findings, and iterative call-graph strongly connected components use captured inputs only. The worker invalidates an older connected cache without this analyzer version. All findings remain qualified candidates.

DPH.Workspace.Comparison compares two hash-verified retained captures from the same project database. It rejects differing selection and syntax or semantic analyzer contracts. Deterministic identity indexes distinguish file changes, symbol additions or removals, captured structure changes and file-only changes. Routine-local flow numbering avoids global node-number shifts. Results use filtered 200-row pages; individual structure displays are limited to 4000 characters with full-content structure hashes and explicit truncation.

Comparison runs on the existing background worker. It reads retained database artifacts rather than current selected source files. Native comparison controls and evidence panes are declared in DPH.UI.Main.fmx. tests\DPH.Tests.Comparison.dpr checks literal-expression changes, unchanged neighboring routines, flow-edge rewiring, identity changes, contract rejection, pages, native panes, worker loading and missing caches.

The Phase 1 traceability ledger preserves the supplied requirements and limitations. Source comparison remains partial until optional historical source retention supports actual source-text differences; captured structural comparison must never be described as reconstructing those bytes. Human business answers and independent first-time-user acceptance remain separate evidence.

## Keyboard inspector implementation and validation

DPH.Workspace.Keyboard projects captured text and binary form IR into source-backed component records. A parent-property index avoids rescanning every node for every component. Rows retain generation, source SHA256, line and byte extents, original property values and explicit unknowns. Numeric shortcut conflicts normalize decimal and hexadecimal codes within each serialized form; sibling TabOrder conflicts stay parent-local. Unique local action declarations carry separate shortcut and source evidence; uncaptured custom, external or inherited actions remain unresolved.

DPH.UI.Main.fmx serializes every inspector control for the IDE and Object Inspector. The Keyboard and focus page filters readable evidence, sorts sibling TabOrder and uses 250-row pages. TDPHKeyboardInspector.ReadSource enforces matching generation, source identity, project containment, text availability and source hash before navigation. Binary forms expose byte evidence without applying it as text lines. No database migration or target execution is required.

Run tools\build-keyboard.cmd elevated for 55 checks covering actual DFM, FMX and binary parsing, action references, conflicts, unknown states, readable key names, source safeguards, pagination and instantiated FMX controls. The fixed bin/keyboard-validation fixture avoids new disposable test folders. Native PaintTo images in docs/phase1-keyboard-workspace.png and phase1-keyboard-compact.png are renders of running DPH controls, not target application screenshots. The suite checks the user catalog and selected fixture bytes remain unchanged. Runtime traversal and accessibility conformance require separate observed tests.

## Command inventory implementation and validation

DPH.Workspace.Commands builds command records from captured component properties in DPH.Workspace.Keyboard. It indexes parent paths and form-local names, distinguishes duplicate or incompatible references, and joins event properties by exact source and byte location to confirmed static semantic binds relationships. Every handler retains its own identity, relation, target symbol, range and source fingerprint. Action handlers and shortcuts remain separate from component declarations. Uncaptured inheritance, custom classes, external names and runtime dispatch remain qualified.

DPH.UI.Main.fmx serializes the Menus and commands controls for IDE editing. The page provides kind and text filters, 250-row pagination, independent handler selection and Explain navigation. Command and action source commands reuse the keyboard inspector generation, hash and containment checks. Loading a replacement capture clears stale command rows. No database migration or target execution is required.

Run tools\build-commands.cmd elevated for 62 focused checks using actual text DFM, binary DFM, FMX and semantic fixtures, including action execute/update, own click and popup handlers, unresolved references, pagination and native control bindings. Fixtures reuse bin/commands-validation. Source bytes and the user catalog are checked unchanged. PaintTo renders in docs/phase1-commands-workspace.png and phase1-commands-compact.png show running DPH controls, not screenshots of the target app. The keyboard suite remains a required regression because both inspectors share captured properties.

## Dynamic control relationships and validation

DPH.Analysis.Symbols captures constructor owner expressions and visual property assignments in reusable routine facts. VisualSites joins them to creation-site identities after cache reuse. Unique same-routine result handles support parent and event declaration references. Multiple constructors, intervening variable writes and assignments before construction prevent attribution and retain ambiguous evidence. Target declarations are inferred lexical references; no runtime binding or reaching-definition proof is claimed. DPHSemanticVersion is DPH-WP3-15 so older semantic captures are reassessed.

DPH.Workspace.DynamicControls indexes the new relationships and retains target symbols, source hashes, ranges, conditions and generation evidence. DPH.UI.Main.fmx serializes the filter, 250-row list, details, relationship selector and Explain controls for the IDE. Changing capture clears stale rows. Missing references disable Explain reference. Creation sites remain distinct from serialized form geometry and observed screenshot evidence. No database migration or runtime UI construction was introduced.

Run tools\build-dynamic-controls.cmd elevated for 51 focused checks, including owner/parent/event references, nil, reused and overwritten variables, pre-creation assignments, unresolved expressions, loop counts, exact fresh/cache equivalence, pagination and native form controls. Fixtures reuse bin/dynamic-validation. The 134-check semantic regression and 62-check command inventory regression pass, as does the actual app and FMX resource baseline. Historical phase evidence is preserved; current semantic evidence is docs/phase1-dynamic-semantics-regression.json. Native normal/compact PaintTo renders show running DPH controls, not target screenshots.

## Layout rule explanations and validation

DPH.Workspace.Keyboard retains captured layout properties from reusable form IR. DPH.Workspace.LayoutRules indexes serialized parents and their properties, preserving source ranges and fingerprints. Describe explains VCL and FMX alignment modes, exact anchor tokens, geometry, spacing,

constraints, autosizing, wrapping, DPI and resize handlers. Missing values and inheritance are qualified rather than filled with assumed defaults. Contradictory captured minimum and maximum constraints produce a review candidate.

Educational numeric examples require explicit recognized anchors and Align=None on an ordinary child with a captured parent. Opposing edges change size by the parent delta; trailing edges change position; leading edges preserve position. Unanchored axes, active or unknown alignment, inherited and root objects suppress unsupported calculations. These are basic rule examples and never an evaluation of the VCL or FMX layout engine or final runtime geometry.

DPH.UI.Main.fmx serializes every new page control for IDE editing. The page provides full-text filtering, 250-row pagination, validated positive or negative deltas, source verification, a horizontal command bar and a resizable scrollable detail area. Replacing a capture clears stale rows. No target classes are instantiated and no database migration or analyzer-version change is needed.

Run tools\build-layout-rules.cmd elevated for 56 focused checks using real text DFM, binary DFM and FMX parsing and native form controls. Tests cover growth and shrinking, active alignment, inherited and missing rules, unknown anchors, constraints, parent properties, navigation guards, pagination, invalid and overflowing input and normal and compact layouts. Fixtures reuse bin/layout-validation. The shared keyboard suite passes 55 checks and dynamic controls passes 51. The actual application and FMX baseline pass. Native PaintTo renders show DPH controls rather than target screenshots.

## Component dependencies and attributed metadata

DPH.Workspace.ComponentDependencies indexes captured semantic class declarations by qualified and simple name, joins them to serialized component classes, and preserves declaration ranges, source hashes, conditions and generation context. Unique matches are inferred candidates rather than compiler binding proof. Ambiguous or absent matches remain explicit. Captured package declarations and unavailable inputs are context only; membership, installed versions and vendor origin are not inferred.

DPH.Knowledge.ComponentDependency validates bounded text fields for class identity, declaring unit, package/product, vendor, version/configuration, support and evidence. DPH.UI.ComponentDependency.fmx serializes the scrollable editor. The main page controls are serialized in DPH.UI.Main.fmx. Apply prepares a structured Knowledge draft; the existing worker saves it with attributed revision history. Storage enforces identity, canonical readable body, case-insensitive uniqueness, stale-edit protection and retention of structured fields. No schema migration or target component loading was added.

Saved dependency records participate in Knowledge search, engineering publication, evidence links and named review. Editing reopens review; retired class identities stay reserved. The native inspector refreshes saved metadata without changing source resolution, protects unsaved drafts, provides 250-row pagination, filtering, source guards and unique-candidate Explain navigation.

Run tools\build-component-dependencies.cmd elevated for the focused acceptance suite. It exercises real text/binary DFM and FMX parsing, unique/ambiguous/unresolved class candidates, captured packages and missing dependencies, SQLite reopen/history/review/search/publication, duplicate and stale edits, native editor round trips, navigation, pagination and compact layout. Fixtures reuse bin/component-dependencies-validation. Native PaintTo images show DPH controls rather than target screenshots. Shared Knowledge, glossary and layout regressions and the actual app/FMX baseline are also checked. See the current completion record for measured counts.

## Resolve form inheritance and frame overlays

DPH.Workspace.FormInheritance resolves an inherited root through its matching Pascal class and captured ancestor relationship to one captured base form. Class-to-form lookup uses the matching Pascal file stem and serialized root class. Qualified frame names disambiguate duplicate simple names. Custom resource mappings and aliases may remain unresolved. Conditional ancestry is not silently selected.

The resolver copies base trees, matches inherited children by parent-local name and class, appends local property histories and expands inline frames independently. Root renaming preserves base children. Properties and events retain exact source/hash/range, condition and generation origins. Collection bodies retain serialized history but collection-item merging is explicitly unresolved. Missing bases, duplicate matches, partial forms, incompatible/orphan overlays and cycles are diagnosed.

The IDE-editable main FMX page provides filter, 250-row pagination, an origin selector, source navigation and a scrollable explanation. Origin source uses the existing generation/fingerprint/root/text checks. Capture replacement clears borrowed rows and origin pointers. No storage schema changes, target instantiation or runtime UI construction were introduced. Existing form-design and other inspector qualifications remain in effect.

Run tools\build-form-inheritance.cmd elevated. Controlled fixtures reuse bin/form-inheritance-validation and exercise actual text/binary DFM and FMX parsing, three-level form inheritance, inherited and separately overridden frames, qualified ambiguity, event nil, conditional/missing ancestry, cycles, provenance, cache equivalence, source safeguards and native compact layout. Work is bounded to 64 levels, 200000 component visits and 100000 output rows; gaps are capped at 10000 per selected form. Exceeded work/output bounds reject the overlay operation explicitly. Native PaintTo renders show DPH controls, not target screenshots.

## Store attributed action purpose mappings

DPH.Workspace.ActionPurposes derives candidate targets from the captured source path and parent-local component path. A case-insensitive SHA256 identity is stable across parser node-number changes. Duplicate paths remain ambiguous and disable Edit and Explain. A unique captured component range supplies its semantic object key; no runtime action or business intent is inferred. Project tasks use a GUID identity and no object link. Unmatched saved UI targets remain historical or unresolved.

DPH.Knowledge.ActionPurpose validates twelve bounded string fields, requires the name, purpose, steps, outcome and evidence, and derives the readable Knowledge body. DPH.Storage.Database preserves the record type, stable target identity, object scope, user_help kind, user document and user audience. Duplicate target records are refused, including retired identities. Ordinary Knowledge editing cannot discard structured fields; stale saves and reviews retain the existing concurrency protections.

DPH.UI.ActionPurpose.fmx and the Action purposes tab in DPH.UI.Main.fmx serialize every native control for IDE editing. Apply prepares the existing Knowledge draft and preserves its exact evidence links; Save persists it. Named review appends a revision; subsequent edits reopen review. Search and publication use the same attributed Knowledge record. Existing storage migrations suffice; no new schema or runtime UI construction is required.

Run tools/build-action-purposes.cmd elevated for parser, binary DFM, FMX, identity and ambiguity checks, database persistence, duplicate and stale protection, named review, history, search, exported bundle validation, HTML and user DOCX text, native editor Apply/Cancel, task drafts, filtering, pagination and

compact layout. Fixtures reuse bin/action-purposes-validation. Native PaintTo images show real DPH controls; they are not screenshots of the analyzed application.

## Compare attributed screen evidence and captured layouts

DPH.Workspace.ScreenHistory verifies managed PNG bytes against their saved image hashes, bounds each image and the union canvas to 16 million pixels with dimensions no greater than 8192 and encoded size no greater than 20 MiB, then decodes with the VCL PNG library on the worker. CPU bitmaps composite transparency on white and compare exact eight-bit RGB. No target form, class or executable is loaded. The report preserves image and metadata hashes, contributors, generation IDs, dimensions, counts and inclusive bounds; private managed paths are omitted.

The screen_history worker command resolves screenshot IDs through the existing checksum-validated store. Optional layout comparison loads each image generation from its retained workspace cache. Missing or incompatible layout evidence yields layout_gap without discarding verified images. Parent-local component paths match captured classes and sorted inspector properties; duplicate identities are refused. Source offsets alone do not count as a change. Comparison supports 5000 components per form; all changes remain in the paged list, while previews draw at most 300 rectangles per side.

DPH.UI.Main.fmx serializes the Screen history page, grid, image and paint controls, selectors, toolbar and details. Panels use client alignment in their grid cells. Source geometry is fitted on a common canvas in blue and orange. Request tokens plus the database identity prevent a late response from restoring evidence after a pair or path change. Processing and PNG comparison run in the existing worker; presentation uses FMX. No storage migration, runtime control construction or source mutation is required.

Run tools/build-screen-history.cmd elevated for 73 checks covering exact pixels/bounds, transparency, size changes, altered bytes, union limits, text and binary DFM, FMX, parser/selection incompatibility, persisted image attribution, missing retained captures, native Compare, late-response rejection, pagination/filtering, swap, visible geometry and compact layout. Fixtures reuse bin/screen-history-validation. The owner catalog is restored after exercising cached open. Native PaintTo renders show actual DPH controls; controlled synthetic images do not establish target-runtime observations.

## Build a safe captured database connection inventory

DPH.Workspace.DatabaseConnections projects captured text/binary DFM and FMX IR directly. A parent-property index joins database configuration and references to each candidate. An explicit class list covers common connection, dataset, data-source, transaction and session declarations; recognized property signals retain custom-class candidates without claiming vendor or runtime behavior. Existing component_field semantic relationships prevent duplicate Pascal field rows. Unmatched supported Pascal fields, variables and locals retain declaration evidence and unknown construction/configuration.

Resolution uses source-local root and parent paths, explicitly qualified cross-form paths, expected target roles and captured logical aliases. There is no global short-name guess. Duplicate targets remain ambiguous. Named ConnectionDefName references share a definition row with explicit unavailable contents. Each row retains generation, source hash and range; exact outgoing and reverse IDs drive native Follow reference navigation. Missing inherited values, runtime assignments, factories and external definitions remain unknown. A 10000-row bound prevents an unbounded inventory.

The projection never copies raw properties or source text into its configuration. Known parameter keys are retained with masked values; unknown keys use a generic label. Endpoints, credentials and entire connection strings remain masked. Only bounded identifiers for driver/protocol/dialect/charset/SSL, valid ports and explicit booleans are visible. Unsupported reference expressions are masked. Original captured source and caches are unchanged and may contain secrets; this feature is not a comprehensive secret-removal claim for the rest of DPH. No target driver, component class or database connection is created.

DPH.UI.Main.fmx serializes every page control for the IDE and Object Inspector: purpose, horizontally scrolling toolbar, kind/text filters, paging, link selector, Follow reference, resizable list, labeled memo and status. LoadView rebuilds the projection for the accepted generation; ClearWorkspace discards borrowed rows before releasing their owner. Follow selects the exact target page after clearing filters. Compact validation requires a visible evidence area. Existing storage schema and analyzer contracts suffice.

Run tools/build-database-connections.cmd elevated for 66 checks using real text/binary DFM, FMX and Pascal declarations, persisted/reopened captures, local/nested/cross-module links, aliases, duplicate and missing targets, sensitive canaries, validated settings, native paging/filtering/follow/empty states, capture replacement, compact layout and unchanged source/owner catalog. Fixtures reuse bin/database-connections-validation. Native PaintTo renders show DPH controls, not screenshots of an analyzed application. Regression evidence also covers 134 semantic/SQL and pinned-reference checks and 73 screen-history checks.

## Derive captured database engine and version evidence

TDPHDatabaseConnections.EvaluateEngines extends the existing bounded inventory. It maps recognized captured DriverID values only for TFDConnection declarations using the FireDAC driver contract; it does not infer engines from class families, SQL tokens, DLL names, ports or DPH internal SQLite. The mapping basis is Embarcadero Databases FireDAC documentation at https://docwiki.embarcadero.com/RADStudio/Sydney/en/Databases_(FireDAC). Supported IDs identify families, without deriving server release or compatibility. MySQL-compatible retains compatible-server uncertainty; unknown virtual drivers are not silently mapped.

Configuration permits bounded numeric ServerVersion, DatabaseVersion, ClientVersion and DriverVersion fields only. Server version aggregation preserves matching declarations, reports disagreement, and never promotes client or driver version evidence. Invalid values remain masked. Each engine fact carries its original source, line/range, source hash and captured generation. External definition contents, runtime overrides and live version measurements remain explicit unknowns. No driver or target database is opened.

A dictionary indexes existing row IDs. Three bounded passes propagate evidence through unique captured connection and dataset targets to datasets, transactions and data sources. Multiple target identities become conflicting; unresolved candidate paths suppress propagation. Original fact ownership remains on the connection even when a dependent row displays it. The existing storage and analyzer contracts suffice; persisted capture reopening deterministically rebuilds the same projection.

DPH.UI.Main.fmx retains IDE-editable controls and adds engine instructions and a concise evidence label. List rows summarize family and declared version. Details show engine results before qualifications, improving compact-window usability. The text filter searches evidence fields and safe configuration, excluding general boilerplate; navigation, pagination, capture replacement and credential masking continue to use the existing inventory.

Run tools/build-database-engines.cmd elevated for 66 focused checks covering all fourteen driver IDs, unknown and virtual drivers, conflicting engine/version declarations, client/server separation, unresolved and conflicting paths, text/binary DFM and FMX, persisted reopening, source hashes, sensitive canaries, native filtering/details and compact renders. Existing database inventory and screen-history suites add 139 regression checks. Actual app and FMX baseline build pass. Fixtures reuse bin/database-engines-validation; native PaintTo renders show DPH controls and controlled declarations, not observed target-runtime connections.

## Inspect captured database schema evidence

DPH.Workspace.DatabaseSchema projects captured SQL resource IR and existing semantic SQL-table references without storage migrations or target execution. A bounded lexer separates comments, quoted identifiers and string literals, preserves statement-start lines, and recognizes supported CREATE TABLE, VIEW and SCHEMA headers. Qualified names retain quoted spelling; two- and three-part names yield qualifier-reference rows. PostgreSQL CREATE TABLE, CREATE VIEW and CREATE SCHEMA documentation provides header references, not a cross-dialect grammar guarantee. Unsupported headers, ALTER and other statements remain explicit gaps.

The lexer limits each resource to 2 MiB characters and 100000 tokens, nested comments to 16 levels and the inventory to 10000 rows. Malformed lexical resources yield a gap instead of partially promoted declarations. Definitions omit comment and string contents and bound display length. Parenthesis balance and recognized headers do not validate body grammar, type or constraint semantics. Whole-resource byte extents and precise statement-start lines are labeled separately. Existing inferred references retain captured semantic ranges and use original relationship labels for object names.

Exact kind and spelling index duplicate declarations projectwide; every captured definition remains visible and migration order remains unresolved. The inspector neither normalizes dialect-dependent identifiers nor selects an effective production definition. References stay distinct from declarations; schema qualifiers stay distinct from explicit schema declarations. Missing resources and unsupported constructs cannot establish an empty live schema.

DPH.UI.Main.fmx serializes the Database schema page, toolbar, filters, list, divider, labeled memo and source command for the IDE. Main owns the projection and borrows only visible rows, clearing them before capture replacement or destruction. Filtering and paging show 250 rows. Open source reuses TDPHKeyboardInspector.ReadSource for generation, containment, identity, decoding and SHA256 guards. No runtime controls or additional navigation helper are introduced.

Run tools/build-database-schema.cmd elevated. Focused checks cover declarations, qualified and quoted names, comments/literals, duplicate definitions, unsupported and malformed resources, bounds, inferred references, SQLite persistence, source guards, unchanged workspace/catalog/source, native filters, pagination, capture replacement and normal/compact PaintTo renders. Run existing engine regression and tools/build-wp0.cmd for app and serialized FMX binding. Fixed ignored bin/database-schema-validation holds controlled practice data. No target SQL, database driver or application is executed.

## Extract captured column declarations and constraint text

TDPHDatabaseSchema.Inspect extends the existing resource-token projection. It splits CREATE TABLE body entries only at outer commas, retaining type precision, CHECK and default-function commas. Supported entries emit column rows with declared_type, nullability, default, constraints, table and declaration_id.

Explicit table-level constraint entries emit captured constraint text rows. These facts describe captured syntax, not a validated live schema. PostgreSQL CREATE TABLE documentation supplies the reference structure; this bounded reader is not a full dialect grammar.

Type text accepts an identifier or qualified type, supported multiword suffixes, parenthesized parameters, time-zone qualifiers and array or unsigned suffixes. Clause extraction distinguishes DEFAULT NULL from explicit NULL, accepts bounded simple or parenthesized defaults, and captures supported named and inline constraints. String contents remain omitted. Token-aware formatting preserves quoted identifier spelling while presenting readable numeric and type evidence. Unsupported clauses, absent types, incomplete defaults, repeated defaults, conflicting NULL/NOT NULL and trailing empty entries remain gaps. A supported later entry does not inherit the preceding gap.

Each column and table-level constraint keeps a unique parent declaration ID plus original file, source SHA256, generation and entry-start line. Duplicate table declarations retain independent column facts instead of selecting an effective migration. Whole-resource byte extents remain explicitly labeled. Key relationships, domain/inheritance rules, expression evaluation, dialect validity and deployed enforcement are unverified. No analyzer schema migration, target driver connection or runtime control construction is introduced.

DPH.UI.Main.fmx adds column/constraint kinds and declared column/captured constraint states to existing Object Inspector editable selectors. Existing filter logic also searches declared type, explicit nullability and inline constraint text. Details lead with column type/nullability/default before long provenance explanations, improving compact-window use. Existing 250-row pages, capture replacement, labeled scrolling memo and guarded source navigation remain in use.

Run tools/build-database-columns.cmd elevated for 121 checks using actual controlled project/resource analysis, SQLite persisted reopening, native FMX filtering/details/paging and normal/compact PaintTo renders. Checks cover type precision, multiword and qualified/quoted types, defaults, NULL separation, named/inline/table constraint text, nested commas, unsupported entries, duplicate declaration ownership, literal canaries and unchanged source/catalog/workspace. Existing schema and engine suites add 123 regression checks. Actual application build and FMX baseline pass. Fixed ignored bin/database-columns-validation contains controlled practice data. No SQL or target application is executed.

## Resolve captured key membership and references

TDPHDatabaseSchema.Inspect emits key rows from supported captured CREATE TABLE entries. Each row preserves key_kind, constraint_name, ordered members, reference_table, ordered reference_columns, referential_actions, table, declaration_id and sanitized definition. Existing file, SHA256, generation and entry-start line provenance remain attached. Inline clauses are collected during column parsing and promoted only after the complete column entry passes its supported grammar, preventing an unsupported later clause from leaking a partial key.

Table constraints accept an optional CONSTRAINT name and PRIMARY KEY, UNIQUE or FOREIGN KEY with nonempty identifier lists. Foreign keys accept a qualified referenced table, optional referenced column list and bounded ON DELETE or ON UPDATE actions. Repeated members, expressions, malformed lists, unknown actions and unsupported tails yield key gaps. CHECK remains captured constraint text. Inline reference actions outside the existing column subset remain column gaps. Token and row bounds remain unchanged. PostgreSQL constraint documentation supplies the reference syntax: https://www.postgresql.org/docs/18/ddl-constraints.html. This source reader does not implement that complete dialect.

ResolveKeys builds exact table and declaration-owned column indexes with multiplicity counts. Each key receives ordered links containing role, reference, position, state and target_id only for unique captured matches. The owning table link uses the exact parent ID. Referenced columns resolve only after a unique referenced table is found. Missing or ambiguous evidence never chooses a first or last declaration. Omitted referenced columns and arity mismatches receive explicit notes; multiple primary keys and duplicate names receive conflict flags. Relationship status describes evidence completeness only, not semantic validity, target eligibility or enforcement.

DPH.UI.Main.fmx serializes the key kind and declared key state plus a relationship selector and Follow relationship button in the horizontally scrolling schema toolbar. Main owns target ID strings separately from borrowed projection rows. Selection and capture replacement clear those strings. Follow resets filters, locates the exact target ID, computes its 250-row page and selects it. Existing source guards handle direct key navigation. No runtime controls, new storage migrations or SQL execution are introduced.

Run tools/build-database-keys.cmd elevated for captured resource and SQLite reopening tests, composite order, named and anonymous inline keys, self references, missing or ambiguous members and tables, omitted target columns, arity differences, repeated columns, duplicate primary keys and names, unsupported syntax, literal omission, guarded source reading and native filters and relationship navigation. The suite also retains prior schema and column checks. Run column and schema regressions and tools/build-wp0.cmd for the actual executable and FMX baseline. Practice data stays in fixed ignored bin/database-keys-validation.

## Capture index structures and SQL reference evidence

TDPHDatabaseSchema.Inspect recognizes bounded CREATE INDEX declarations from already captured SQL resources. Supported headers retain explicit index names, optional UNIQUE, CLUSTERED or NONCLUSTERED, CONCURRENTLY, IF NOT EXISTS, ON ONLY and a qualified target table. Optional USING retains the declared method. The reference structure is PostgreSQL CREATE INDEX: https://www.postgresql.org/docs/18/sql-createindex.html. Clustering keywords also retain captured structure from the supported cross-dialect subset; DPH does not validate combinations against a selected database dialect.

Ordered members retain position, column when it is a plain identifier, sanitized expression text, explicit sort and null order. Parenthesized and simple function expressions remain unresolved dependencies. INCLUDE accepts distinct plain column identifiers. Optional unique null treatment, balanced WITH option text, TABLESPACE and balanced WHERE predicate text are retained without evaluation. Empty lists or expressions, malformed name/list structure, unsupported member options or clause tails, excessive nesting and unsupported predicate subqueries remain gaps. Existing lexical, resource-size, token and row bounds apply; index expression nesting is limited to 128.

The existing relationship projection indexes exact table names and declaration-owned columns, using multiplicity counts. Index links target only unique captured table/column evidence. Duplicate table targets, columns and same-table index names remain explicit ambiguities or conflicts. Index records retain source SHA256, generation, statement line, resource byte extent, exact name and table scope. Effective migration order and live schema remain unknown. No schema migration, driver connection, SQL execution or runtime UI construction is introduced.

A separate exact-name lookup groups existing supported SQL table-reference rows. Index records show total and displayed reference counts and link to those captured evidence rows, capped at 250 references per index and 10000 across the projection. These links are labeled index use unobserved. They do not imply

column coverage, plan selection or measured benefit. Absent supported matches cannot establish an unused index; quoted names and case are not normalized across dialects.

DPH.UI.Main.fmx adds index and declared index to the existing Object Inspector editable selectors. Main reuses the relationship selector, exact-ID Follow relationship navigation, 250-row paging, capture lifecycle and source guards. Search includes target table, method, predicate, storage kind and options. Details lead with index facts and explain evidence limitations before an investigation of performance can be justified.

Run tools/build-database-indexes.cmd elevated for actual project/resource capture, ordered composite and expression members, uniqueness and optional structure, literal omission, missing or ambiguous target evidence, malformed/unsupported grammar, persisted SQLite reopening, unchanged source/catalog/workspace, guarded source reading and native filtering/follow/replacement/normal/compact renders. The suite retains prior schema, column and key checks. Run key and schema regressions and tools/build-wp0.cmd for the application and serialized FMX bindings. Controlled practice stays in fixed ignored bin/database-indexes-validation.

## Capture stored-object definitions and dependencies

TDPHDatabaseSchema.Inspect projects supported stored-object declarations from existing captured SQL IR into procedure, function and trigger rows. CREATE, CREATE OR REPLACE and CREATE OR ALTER retain their header operation; PROC normalizes to procedure. Explicit qualified and quoted names are retained without dialect case folding. Named parenthesized parameters and supported TSQL @parameter lists retain order, modes, declared type, sanitized defaults and source lines. Scalar/qualified type structure, numeric precision, supported multiword types and array suffixes are captured; unsupported structures remain extraction gaps.

Inline BEGIN/END blocks retain internal semicolons and nested BEGIN/CASE structure, with a 128 nesting limit. Their internal DDL is never promoted to a top-level table declaration. An unterminated block holds the remaining resource as a gap. Standalone GO client batch separators preserve following declarations. Single-quoted and dollar-quoted SQL or PLPGSQL bodies are lexed again for sanitized display and supported dependency evidence; comments and literal values remain omitted. Other quoted-body languages, unsupported tails, unnamed parameters, table-valued return structure and custom terminator directives remain gaps. Capturing a supported header/body extent does not validate body grammar.

Trigger structure retains supported BEFORE/AFTER/INSTEAD OF timing, INSERT/UPDATE/DELETE/TRUNCATE event lists, ON table, optional FOR EACH ROW/STATEMENT and parenthesized WHEN text. Supported bodies are inline blocks or explicit zero-argument EXECUTE FUNCTION/PROCEDURE calls; the latter names a trigger function. Other trigger dialect arrangements or call arguments remain explicit extraction gaps. Body text and WHEN are unevaluated.

Dependency extraction records supported FROM/JOIN/UPDATE and INSERT INTO literal table names, CALL/EXEC and EXECUTE PROCEDURE targets, plus event tables and explicit trigger functions. It is textual evidence with exact source lines, not a complete semantic SQL dependency graph. CTE/WITH scope, dynamic execution and complex table expressions retain unresolved evidence. No alias, expression-function call or column binding is invented. Projection-wide limits are 1000 dependency entries per stored object and 10000 total, alongside existing 2 MiB resource, 100000-token and 10000-row limits. Body display is limited to approximately 16000 characters and definition display to approximately 8000.

ResolveStored builds multiplicity indexes for captured table/view names and kind-specific routine names. Unique exact name matches receive target IDs with binding explicitly unverified. Missing or ambiguous

declarations and overloads receive states without targets. Duplicate definitions remain separate and effective migration order remains unknown. Stored rows carry resource SHA256, generation, statement line and resource byte extent; parameter/dependency lines refer to that same captured resource. The captured workspace, source files and owner catalog remain unchanged.

The IDE-editable Database schema selectors add procedure/function/trigger and declared stored object. Existing exact-ID Follow relationship, paging, labeled scrolling details and source hash/generation guards are reused. Search includes parameter JSON text, language, returns, trigger facts and sanitized body. No runtime controls, driver connections, server SQL execution or new helper unit are introduced.

Run tools/build-database-stored-objects.cmd elevated for actual project/resource capture, parameters/defaults/types, quoted bodies, triggers, nested blocks, explicit calls, missing/ambiguous targets, overloads, literal omission, limits, batch boundaries, persistence, unchanged source/catalog/workspace, source guards and native navigation/filter/replacement/normal/compact renders. Run index and schema regressions and tools/build-wp0.cmd for the actual application and FMX resource bindings. Controlled fixtures stay in fixed ignored bin/database-stored-objects-validation.

Primary syntax references: PostgreSQL CREATE FUNCTION https://www.postgresql.org/docs/18/sql-createfunction.html, CREATE TRIGGER https://www.postgresql.org/docs/18/sql-createtrigger.html and Microsoft CREATE PROCEDURE https://learn.microsoft.com/en-us/sql/t-sql/statements/create-procedure-transact-sql. The parser implements a bounded captured-evidence subset; these references do not imply full dialect validation or deployed behavior.

## Project captured entity relationship metadata

TDPHDatabaseSchema.Diagram projects independent JSON from the existing captured schema inventory. Only declared table headers become nodes. IDs, exact ownership, source SHA256, generation and source ranges survive projection. Owned columns retain order, declared types, nullability and resolved local key membership badges. Duplicate badge kinds are condensed. No source, owner catalog or persisted workspace is changed.

Captured foreign keys become numbered edges with exact key, owner and referenced table IDs, ordered members and referenced columns, source provenance and relationship state. A solid blue line requires complete captured key links and both visible table endpoints. Unresolved captured links with available endpoints draw orange dashed lines. Missing targets receive no invented ID or ghost node. Relationships touching either visible endpoint remain in the evidence selector when the other endpoint is filtered or off page. Arrow direction runs from the referencing table to the referenced table; no crow-foot cardinality is inferred.

Focus uses the selected declaration ID and a one-hop incoming/outgoing neighbor set. It does not expand transitively. Text filtering applies after focus and stable inventory order is retained across pagination. Bounds require a nonnegative offset, 1 to 60 tables and 1 to 32 summarized columns; native page choices are 15, 30 and 60. The projection retains at most 120 related foreign keys with total and omitted counts. Duplicate names, owned gaps, unresolved keys, absent columns and truncated summaries mark node notes and orange card borders. Global qualification remains PARTIAL CAPTURED SCHEMA even for supported individual declarations.

DatabaseERTab and all command, scroll, memo and style controls are serialized in DPH.UI.Main.fmx for the IDE Object Inspector. The existing Main unit paints the data using a native canvas; it does not construct

runtime controls or introduce a helper unit. Table size, header height, column row height, gutters, margins, fonts, colors and zoom controls are editable form properties. DatabaseERRect arranges stable table cards in a grid; orthogonal FK routes use outside-card gutters, arrowheads and selector numbers, including self links. The canvas resizes for content and magnification. Defaults summarize seven columns.

Table hit testing and FK selection retain exact evidence IDs. Open evidence clears schema filters and navigates to the correct 250-row inventory page. Focus accepts a selected table or the declaration owner of a column/key/gap. Capture loading rebuilds the projection and resets stale focus/evidence IDs; empty results explain recovery. Drawing and details scroll independently, commands scroll horizontally, and a serialized splitter adjusts evidence space. Open source remains subject to the existing generation/hash guards on the schema page.

Run tools/build-database-er.cmd elevated. The suite captures real project SQL resources and checks projection ownership, badge membership, provenance, stable order, limits, missing/duplicate/self targets, focus, filtering, paging, immutable source/catalog/workspace and SQLite reopening. Native tests exercise card selection, exact evidence navigation across pages, the serialized page selector, zoom, compact scrolling and capture replacement; screenshots include the full native canvas and actual window. Fixtures use the fixed ignored bin/database-er-validation folder. Run stored-object and key regressions and tools/build-wp0.cmd for the actual application/FMX resource baseline.

The visualization proves supported captured structure, not a live schema, SQL semantic validation or business meaning. Dialect validity, effective migration order, referenced uniqueness, type compatibility, cardinality and deployed enforcement remain unverified. Views and inferred usage references remain schema evidence rather than invented entities. Source gaps and display limits stay visible. No target SQL, target application or database server is executed.

## Maintain attributed schema dictionaries

TDPHDatabaseSchema provides NewDictionary, ValidateDictionary, DictionaryIdentity, DictionaryTitle, DictionaryBody and FindDictionary in the existing schema unit. A data_dictionary record contains fourteen bounded text fields. Its captured anchor retains the exact schema record ID, source SHA256, original capture generation, kind, name, owning table and statement line. Authored fields hold business definition, owner or steward, sensitivity/handling, evidence/unknowns and optional units/usage. Validation requires explicit content for the first four authored fields; Unknown is a legitimate attributed answer, not an inferred fact.

DPH.UI.DatabaseDictionary.pas and its FMX resource define a native structured editor. All labels, memo rows, dimensions, read-only states and Apply/Cancel controls are serialized and editable in the Object Inspector. Business fields appear first, captured identity below. DPH.UI.Main reuses the existing Knowledge draft/save flow, contributor, immutable revision history, optimistic revision checks, pending review and rejection/retirement commands. No new helper unit, runtime controls or database migration is introduced.

SaveKnowledge validates new anchors against the selected persisted workspace using the captured schema inventory. Forged object names or source lines are rejected. Scope, title, deterministic rendered body and target identity cannot be altered on an existing record. One dictionary per captured anchor is enforced, including retired records. Changes create a fresh needs_review revision. Contributor, editor and reviewer remain independent. Applying the native editor only prepares a draft and requires an explicit Save.

Database schema details show business meaning before captured technical facts. Its filter searches current authored dictionary bodies. Exact captured bytes are required for current lookup; old hash matches are

reported as historical rather than moved onto a new source version. Unmatched historical records remain accessible in Knowledge. DPH.Publish.Model filters dictionary sections by included source files and matching SHA256, preserving existing publication redaction and attributed review state. Publisher version DPH-LEARNING-9 invalidates prior publication fingerprints.

Run tools/build-database-dictionary.cmd elevated. Its fixed ignored bin/database-dictionary-validation fixture captures real SQL resources, persists the workspace, checks required fields/types/bounds, forged/duplicate anchors, immutable identities, conflicting body rejection, contributor/review/revision history, stale review, table/column separation, historical lookup, export exclusions/hash mismatch, native serialized controls, modal Apply-to-draft flow, search, history navigation, compact renders and SQLite reopening. No selected application or SQL executes. Run ER, publishing and glossary regressions plus tools/build-wp0.cmd for the actual application and FMX baseline.

The editor implements an attributed dictionary workflow, not automatic knowledge of business meaning. Sensitivity is descriptive text, not an authorization mechanism. The captured schema remains partial and does not establish live deployment, SQL validity, migration order or business correctness. Human review and the source evidence must qualify every business assertion.

## Resolve captured dataset and field bindings

TDPHDatabaseConnections.Inspect extends the existing captured form inventory with known persistent TField classes, data-aware control property signals, TBindSourceDB and visual controls explicitly referenced by serialized binding Control properties. Only required reference/configuration metadata enters the projection; captions, credentials and connection values retain existing redaction. The source and persisted workspace remain unchanged.

Persistent field DataSet ownership is derived from its exact serialized parent path and resolved only to a unique dataset. Explicit LookupDataSet references remain distinct from ownership. ResolveFields indexes captured IDs and dataset-scoped case-insensitive FieldName values, then joins resolved control DataSource or BindSource, intermediary DataSet and persistent field identity. Multiple fields, incomplete or competing chains receive no target ID. Fields absent from captured declarations are not invented. Every property link retains its original property line and byte range; binary form evidence has byte positions rather than invented text lines.

The existing Follow reference workflow navigates outgoing and reverse references, resets filters and finds exact IDs across 250-row pages. Serialized kind choices add bound control, persistent field and binding control target. Binding explanations and field metadata appear before lengthy engine qualifications. No runtime controls, helper unit, target component loading or storage migration was introduced. Existing engine derivation remains unchanged.

The installed RAD Studio Data.Bind.Components source and official TLinkControlToField documentation establish the published DataSource, FieldName and Control contract. Data.Bind.DBScope supplies TBindSourceDB. Support describes serialized lexical candidates; it does not evaluate LiveBindings expressions or instantiate the target. General expression/factory/runtime/inherited behavior remains qualified rather than promoted to confirmed static binding.

Run tools/build-dataset-bindings.cmd elevated for 92 checks. Real text/binary DFM and FMX fixtures cover VCL and FMX field chains, parent ownership, reverse links, duplicate fields, missing/nil sources, lookup datasets, exact property evidence, native kind/text filters, follow navigation, compact layout, SQLite

reopening and unchanged source/catalog. Connection and engine regressions each pass 66 checks. tools/build-wp0.cmd builds the actual application and verifies FMX resource bindings. Fixtures use fixed ignored bin/dataset-bindings-validation.

## Recognize captured database operation requests

TDPHSemanticAnalyzer.RoutineFacts emits database_operation relations for recognized methods on declared framework datasets or captured descendants. Receiver resolution and imported framework identity are checked; a local class shadowing TFDQuery and ordinary Delete methods are excluded. Captured inheritance is bounded to 32 steps. Within-with calls, factories, aliases, indexed receivers, unsupported providers and runtime dispatch remain unproved. DPHSemanticVersion DPH-WP3-15 invalidates older semantic caches.

TDPHDatabaseConnections.AppendOperations projects API calls, static_sql and dynamic_sql relations into existing database inventory rows. It preserves generation, condition, source SHA256 and line/byte range. Semantic component_field relationships connect operation receivers to serialized components by exact source location; ambiguous locations never choose an arbitrary target. Recognized custom dataset declarations receive their own navigable receiver row. SQL literal values are omitted. Single lexical CRUD families can be retained for complex SQL without inventing table or column bindings; dynamic or mixed batches remain unknown.

Effects distinguish record-buffer preparation, posting, cached-update application, deletion requests, SQL configuration and execution requests. Engine evidence follows captured receiver references. The database operation kind filter and Source button are serialized in DPH.UI.Main.fmx. Existing pagination, reverse references and source hash guards are reused. No helper unit, runtime control, storage migration, target execution or driver loading is introduced.

Run tools/build-database-operations.cmd elevated for 232 checks using the actual analyzer, persisted/reopened SQLite workspace and native FMX form. Fixtures cover CRUD requests, local/custom datasets, framework-name shadowing, SQL joins and mixed/dynamic SQL, operation effects, exact receiver/source identity, redaction, reverse navigation, detached-source refusal, compact layout and unchanged source/catalog. Binding regression passes 92, engine regression 66 and semantic regression 134 checks. tools/build-wp0.cmd builds the real application and verifies its FMX resource. Fixtures stay in fixed ignored bin/database-operations-validation.

## Trace captured transaction boundaries

DPHSemanticVersion DPH-WP3-15 invalidates older captures. RoutineFacts emits database_transaction relations for recognized FireDAC, ADO, dbExpress, BDE and IBX transaction APIs on declared imported framework types or captured descendants. IsDatabaseType shares the existing bounded ancestry and framework-name shadow checks with dataset operation recognition. The API list is provider-specific; ordinary methods and local classes sharing framework names are excluded. Parameter values and savepoint names do not enter the inventory. No target class, SQL or server is loaded or executed.

AttachTransactionPaths projects existing captured flow_nodes and flow_edges without evaluating expressions. Each boundary maps to the smallest containing statement range in the same routine and source. Separate finalizer copies retain their different pending transfers. Bounded traversal stops at the next boundary on the same declared semantic receiver or a normal/exception exit. Exception and

conditional/loop flags stay separate. Multiple transaction calls in one expression receive no invented ordering. Receiver aliases and interprocedural transactions remain unproved.

Indexes group nodes by routine, outgoing edges by node and transaction boundaries by their statement. Limits are 10000 candidate CFG nodes per routine, 5000 states and 64 routes per boundary, and 1000000 states per inventory. Missing or bounded flow remains explicit. Routes expose node IDs and traversed edge-kind sets for comparison with the underlying CFG; they are not execution traces or path-feasibility proofs. Existing reference navigation links receivers, next boundaries and their reverse references. Controls remain serialized and IDE editable in DPH.UI.Main.fmx.

Calls immediately after try, except and finally now retain their no-parentheses statement-call recognition. Run tools/build-database-transactions.cmd elevated for 402 checks using actual analysis, persisted/reopened SQLite and native FMX controls. Fixtures cover normal/error commit and rollback paths, finalizers, conditional alternatives, missing boundaries, distinct receivers, retaining/savepoint variants, five framework families, captured descendants, local name shadows, unordered expressions, absent CFG, redaction, native navigation, compact layout and unchanged source/catalog. Regression suites cover 232 operation, 92 binding, 66 engine and 134 semantic checks. tools/build-wp0.cmd builds the actual application and checks the FMX resource. Tests reuse fixed ignored bin/database-transactions-validation.

## Capture field validation and transformation evidence

DPH-WP2-6 recognizes serialized BeforePost, BeforeInsert, BeforeEdit, BeforeDelete, BeforeCancel and BeforeOpen as event bindings in both text and binary forms. Existing own-form method resolution supplies handler relations; DPH-WP3-15 invalidates older semantic captures. Existing On-prefixed events retain their previous behavior. Missing or ambiguous method targets remain unresolved.

AppendDataRules in DPH.Workspace.DatabaseConnections indexes captured symbols, event bindings and routine CFG nodes. It projects persistent-field constraints, recognized field/dataset/input hooks and bounded handler statements. A transformation requires two direct expressions in the matching Pascal statement IR, rather than a semicolon heuristic. Conditions, raises and early exits remain distinct. Repeated finally copies are deduplicated by source location and context. Statement input/output links use captured lexical reads/writes within expression ranges, not reaching-definition or runtime-value claims. Literal strings, comments and character codes are omitted; supported custom/default expressions retain their structure without evaluation.

The combined inventory is bounded to 10000 rows; truncation is explicit in the model and native status. Source, SHA256, generation, context, handler, input/output and reverse references retain exact captured identity. Native data rule and rule declaration filters are serialized in the FMX resource. No new UI controls, helper unit, migration, target instance, driver or server execution is introduced. tools/build-data-rules.cmd checks real parser output, persisted/reopened captures, constraints, predicates, transformation inputs/outputs, missing handlers/syntax, semicolons in call literals, redaction, inventory limits, native navigation and normal/compact layouts. Explicit directions are in docs/guides/Data_Rules_Run_and_Test.txt.

## Connect source evidence into scoped lineage

TDPHDatabaseConnections.Lineage projects the captured workspace and database inventory without modifying either. Exact component_field source locations map semantic receivers to serialized inventory identity only when unique. Existing rule declarations retain their identity. Directed derives_from relations connect input candidates to dependent declarations and file output artifacts. Lexical rule inputs and outputs

connect through the captured rule. Serialized bindings, handler context, reads/writes, rendering requests and database operation routines remain labeled context rather than reaching-definition evidence.

The projection indexes at most 50000 symbols and 100000 deduplicated edges, then performs bounded adjacency traversal. Each returned node retains its parent hop, kind, source, SHA256 and capture generation. Edges retain original range, source hash, condition, category and qualification. Both direction traversal includes context; Inputs and Destinations exclude broad procedure context and respect dependency direction. Cycles are deduplicated. Returned limits are 200 nodes, 800 edges and 12 hops. Missing targets, missing dataset declarations, unresolved handlers, scoped connection boundaries and limits remain explicit. Runtime values, aliases, server effects and final success remain unverified.

ResolveFields now treats any captured data-source chain without a dataset target as incomplete, including the case where another source chain resolves. It no longer invents a persistent-field target from the remaining valid branch. Existing exact-role and ambiguity checks remain in force. No analyzer version or schema migration is needed because lineage projects existing captures; older captures without a recognized event or relation still require Full reassessment.

The direction selector, Trace lineage and Lineage diagram buttons and Data lineage graph choice are serialized in DPH.UI.Main.fmx. Main form navigation reuses the inventory and explanation pages; graph grouping is disabled for lineage. GraphHTML renders the same evidence JSON with selectable nodes and unresolved breaks. Existing browser allowlisting and source hash guards remain in force. Tests use actual parsing, persisted and reopened SQLite captures, directional assignment chains, cyclic assignments, controlled file-writer evidence, redaction, incomplete bindings, trace limits and native FMX navigation/layout. No selected target, SQL, driver or production server is executed. Build with tools/build-data-lineage.cmd.

## Project captured migration and version evidence

TDPHDatabaseSchema.Inspect appends migration evidence after ordinary schema identities and links are resolved. Existing schema IDs remain stable. Each captured readable SQL resource becomes a migration candidate; unavailable literal SQL loader targets become migration gaps citing the Pascal loader range and fingerprint. The existing bounded project inventory discovers referenced files only. No unrelated directory scan, migration runner, target database or target application executes.

Leading numeric filenames are order candidates scoped to the captured source directory. Leading zeros normalize only for conflict detection; original filename digits remain visible. Captured loader configures relationships retain their source, line, fingerprint and enumeration label. Neither filename nor lexical-loader order becomes deployment precedence. Schema evidence links retain exact row IDs and are limited to 250 per script, with total and omission counts.

The existing SQL lexer separates comments and literal contents. Supported top-level PRAGMA user_version assignments capture nonnegative 32-bit integer values through equals or parentheses and preserve explicit schema qualifiers. Queries, engine schema_version counters, unsupported expressions, nested procedure blocks and comments cannot become application-version assignments. Multiple assignments remain ordered evidence rather than a selected final version. SQLite reference: https://www.sqlite.org/pragma.html#pragma_user_version. Supported verification request spelling captures integrity_check, quick_check and foreign_key_check without execution or invented results.

Existing SQL bounds are 2 MiB characters, 100000 tokens and 10000 combined schema rows. Migration detail bounds are 250 displayed version assignments, 8000 characters of verification requests and 1000 loader entries, with explicit truncation. IDE-editable kind/state selectors, paging, evidence memo, source guards and relationship navigation are reused. The focused tools/build-schema-history.cmd suite exercises actual parsing, persisted reopening, numeric-prefix conflicts, missing sources, version/counter distinctions, nested blocks, redaction, limits, native filters and navigation. Shared schema, ER, stored-object and lineage regressions verify the same projection remains compatible.

## Project captured file formats and contracts

TDPHWorkspaceView.FileFormats projects existing IR and semantic captures without a schema or analyzer change. Supported captured JSON, XML and INI resources show member shapes while omitting scalar values. Member names are bounded and unsafe names are omitted. Each resource shows at most 250 members and an explicit total/limit. The combined inventory is capped at 10000 rows with visible truncation.

Outer expression tokens identify dotted ReadAllText/Bytes/Lines, WriteAllText/Bytes/Lines, LoadFromFile and SaveToFile spellings. Quoted strings remain single tokens and comments do not become APIs. A supported literal first argument contributes only a safe filename extension. Explicit TEncoding argument spellings remain candidates; byte APIs remain binary candidates. Enclosing captured routine identity provides a producer or consumer link. Evidence retains source hash, generation and enclosing expression range; custom receivers and runtime dispatch are unverified.

DPH.UI.Main and its serialized FMX resource provide kind/text filters, 250-row paging, labeled evidence, guarded source opening, routine explanation and contract drafts. No runtime controls, helper unit or target execution is introduced. Deterministic contract titles include the exact row and source hash. Existing attributed Knowledge storage supplies immutable revision history and separate review. A contract draft includes encoding/BOM, schema and required fields, producer/consumer mappings, validation, compatibility, verification and remaining unknowns.

Run tools/build-file-formats.cmd elevated for the focused actual-parser, persistence and native FMX checks. It uses fixed ignored bin/file-formats-validation fixtures and verifies redaction, routine identity, explicit encoding candidates, resource/transfer distinctions, large shapes, paging, saved contracts, source-change and detached-source refusal, compact layout and unchanged captured source/catalog. Shared schema-history, lineage, Knowledge and publishing regressions and tools/build-wp0.cmd validate integration. Candidate evidence and authored contracts do not establish a formal schema, runtime transfer success or deployed compatibility.

## Maintain attributed target retention requirements

Requirement 05.18 is an attributed manual-capture workflow. TDPHWorkspaceView provides the target-policy template, typed-record recognition, metadata validation and readable evidence. Native DataRetention controls in DPH.UI.Main.fmx provide state/text filtering, bounded 250-row pages, labeled scrollable details, draft preparation and existing Knowledge/link navigation. Controls remain editable in the IDE. No helper unit, analyzer pass or storage migration is introduced.

Structured metadata contains record_type data_retention and a stable GUID policy_id. The existing attributed body remains editable; identity and kind stay protected. TDPHKnowledgeStore.SaveKnowledge validates metadata, rejects the default title, anonymous authors, removed heading, wrong business-rule/maintenance/maintainer scope, duplicate identities, identity changes and discarded structured

metadata. Existing optimistic revisions, named review, immutable history and exact rich links remain authoritative. Unknown fields are deliberately permitted and visibly unresolved; policy prose is human evidence, not an inferred legal rule.

The register distinguishes original contributor from current revision editor and shows the named reviewer and explicit link count. Existing Knowledge links support captured semantic routines, evidence and exact procedure-section revisions. Publication includes these requirements and typed identities in maintenance/all-reader handoffs through existing audience and redaction rules. No source application, SQL, deletion or archive job executes. DPH snapshot controls are distinct from the selected application's retention requirements.

Run tools/build-data-retention.cmd elevated. The focused suite uses a real parsed controlled Retention.dpr capture, persisted attributed policies, metadata rejection cases, immutable review/edit history, routine and procedure links, canonical/HTML export and complete bundle validation, native paging/filtering, draft guards and compact rendering. Knowledge, publishing and file-format regressions and tools/build-wp0.cmd cover shared paths. Controlled outputs remain under ignored bin/data-retention-validation; no owner project source or catalog is modified.

## Maintain target data quality evidence and report imports

TDPHWorkspaceView provides NewDataQualityRule, typed record recognition, metadata validation and attributed details in the existing model unit. Stable record_type data_quality and GUID rule_id identify each rule or imported report. Native controls are serialized in DPH.UI.Main.fmx for IDE editing: state/text filters, 250-row pages, command bars, contributor/context inputs, labeled evidence, New/Open, implementation links and import. Main guards unsaved Knowledge drafts and reuses existing revision, review and rich-link workflows. No helper unit, storage migration or target execution is introduced.

TDPHKnowledgeStore.SaveKnowledge validates quality identity, heading, business-rule kind and project maintenance/maintainer scope, rejects placeholder titles, duplicate identities, changed identities and discarded structured metadata. The body remains editable and exact prior revisions remain in history. ImportDataQualityReport reads only explicitly selected .txt/.log/.json files, checks both before/after read sizes at 32 KiB, requires round-trip UTF-8 and rejects empty/NUL text. It hashes the original bytes and saves literal text with contributor, context, UTC, generation and an explicit unverified-claim qualification through SaveKnowledge.

The import_data_quality worker command opens the selected store using its actual source root, imports on the existing background writer and returns saved Knowledge. Report imports do not execute a parser for report commands, SQL, scripts or target code. Publication uses the normal maintenance scope and escapes report markup in HTML; imported measurements remain unverified even when hash-consistent. Exact section links associate rule, operator procedure and imported report revisions; entity/evidence links preserve captured implementation references. DPH storage integrity and selected-data quality remain distinct.

Run tools/build-data-quality.cmd elevated for real parsing/persistence, identity and invalid-save cases, immutable review/edit history, implementation/procedure/result links, native filtering/paging/draft guards, compact rendering, UTF-8/size/NUL rejection, report hashes, literal HTML, full handoff validation, unchanged source/catalog and actual background import. The worker test releases the test writer before opening the background writer. Knowledge, publishing, retention and actual application/FMX regressions cover shared paths. Fixed ignored bin/data-quality-validation holds controlled fixtures.

## Maintain target backup and restoration evidence

TDPHWorkspaceView provides the recovery template, typed metadata validation, recognition and readable attribution. record_type target_recovery and a stable GUID plan_id identify each plan independently of its editable body. TDPHKnowledgeStore.SaveKnowledge enforces the first heading, descriptive stable title, project business-rule and maintenance scope, unique identity and existing optimistic revisions. Named reviews and edits preserve exact prior bodies and evidence links; unknowns remain explicit.

TargetRecovery controls in DPH.UI.Main.fmx are serialized for IDE and Object Inspector editing. The register provides state and text filters, 250-row pages, labeled scrolling evidence, New/Open and the existing exact-link editor. It borrows saved Knowledge rows and clears them before replacement; it guards unsaved drafts. No helper unit, migration, runtime control construction or target backup/restore execution is introduced.

The controlled tests parse a real Recovery.dpr and RestoreOrders routine, persist requirements and attributed test measurements, exercise invalid/duplicate/stale saves, immutable review/edit history, source and exact procedure links, canonical and escaped HTML publication, full bundle validation, native editing/navigation/filtering/paging and compact layout. Knowledge, publishing and data-quality regression suites and the actual app/FMX baseline cover shared paths. Run tools/build-target-recovery.cmd elevated. Fixtures remain under fixed ignored bin/target-recovery-validation.

DPH portable database backup and selected-application recovery requirements are separate. Written measurements are contributor claims, and a stored hash does not authenticate their origin or prove recovery. Existing section links can pin a saved evidence revision. The embedded-NUL report rejection fixture uses an ordinary filename, embedded-nul.txt, to avoid Windows device-name behavior while testing the actual NUL bytes.

## Maintain structured business process records

TDPHWorkspaceView provides the typed business_process object with stable process_id GUID and eight string fields. Validation rejects missing, unexpected, non-text, empty and oversized fields, and an unspecified process name. SaveKnowledge enforces project business-rule maintenance scope, a name matching the stable title, a canonical generated body, unique identity and optimistic revisions. Existing metadata conversion/discard guards, immutable review/edit history and exact evidence links remain authoritative.

BusinessProcess controls and the Process fields editor are serialized in DPH.UI.Main.fmx for IDE and Object Inspector editing. A scrolling native field editor prepares the existing Knowledge draft rather than silently saving. Named contributor, draft guards, editable business fields, protected saved name, separate review, filters, 250-row paging, source explanation navigation and exact section revisions share the existing workflow. No helper unit, schema migration, runtime control construction or target execution is introduced.

Run tools/build-business-process.cmd elevated. The controlled fixture parses Operations.dpr and FulfilOrders, persists and reopens typed fields, tests malformed metadata, duplicate/changed identities, canonical-body divergence, stale saves, named review and immutable history, exact source/procedure links, canonical publication and full bundle validation. Actual native FMX editor/register renders cover normal and compact windows and the last field. Shared knowledge, publishing and data-quality suites and tools/build-wp0.cmd cover regressions. Fixtures use the fixed ignored bin/business-process-validation directory.

Business owner and contributor are distinct. Unknown is an explicit unresolved value rather than inferred business knowledge. Source links establish captured implementation evidence, not that a business step ran.

Generic ordered step text in this inventory does not complete the separate departmental workflow or end-to-end process-diagram requirements.

## Maintain ordered departmental workflow evidence

TDPHWorkspaceView provides departmental_workflow metadata, canonical workflow GUID, five workflow fields and an ordered steps array. Each step has a canonical GUID, department, role, action, inputs, outputs, handoff, exception handling and exception destination. Validation rejects duplicate identities, dangling destinations, unknown shapes, invalid types, empty or oversized fields, unspecified actions, missing steps and exceeded limits. Normal authored order follows the array; exception identity remains stable across reordering.

SaveKnowledge requires project-scoped maintenance business-rule records, stable title matching the structured name, canonical generated body, unique workflow identity and optimistic revisions. Existing immutable save/review history, source invalidation, exact section/source links, named contributors and reviewers, publication and reopening apply. No new schema or helper unit is added. Human workflow content and named review do not establish target execution, business authorization or successful outcomes.

The register and workflow/step fields are native serialized controls in DPH.UI.Main.fmx. New/Edit create a local structured draft; Apply step modifies that draft; Apply workflow prepares Knowledge; Save persists it. Shared nonmodal editor guards prevent saving unapplied fields and bind edits to the original record. New/discarded Knowledge frees its workflow draft. Compact commands scroll horizontally and fields vertically. Array replacement/reordering uses the RTL TJSONArray.SetElements ownership contract while retaining or freeing element values appropriately.

Run tools/build-departmental-workflow.cmd elevated. The real parser, SQLite reopening, revision/review and canonical publication fixture covers ordered Sales/Warehouse steps, roles, handoff, recovery destination, malformed records, invalid identities, dangling/duplicate destinations, bounds, source/procedure links and unchanged source/catalog. Native FMX exercises add/apply/remove protection, reordering, dirty-step and unapplied-workflow guards, editor/register navigation and compact renders. Compatibility covers Knowledge, publishing, data quality and the business-process editor plus the actual application/FMX build. Fixtures remain in the fixed ignored bin/departmental-workflow-validation directory.

This implements departmental workflow capture. End-to-end process diagrams now provide a distinct authored workflow projection; technical graphs retain their source-derived meaning.

## Maintain end-to-end process projections

TDPHGraphProjection.Workflow derives bounded ordered workflow-step nodes, one End node, explicit unresolved exception destinations, normal handoff edges and stable exception edges from a saved departmental_workflow record. No runtime trace, inferred step or invented process association is added. Links resolve against exact in-scope business-process history revisions and retain state, named reviewer and process identity.

TDPHGraphProjection.WorkflowSVG creates accessible, scrollable SVG with blue normal routes, orange exception gutters, escaped labels, full fields/routes and exact process revision links. The native serialized GraphPaint provides a scrollable fallback, separate arrow routing and step details. GraphCombo has an Object Inspector editable Business process choice, and Departmental workflows has a serialized Process

diagram button. Knowledge replacement rebinds workflow rows before graphics refresh to avoid stale pointers.

The version 10 publication model derives process_diagram after audience, source and link scope filtering and redaction. HTML renders it within the workflow article. Canonical knowledge JSON and its existing index retain it. Validator recomputes the projection and rejects divergence or missing diagrams in current-template workflow records. Localization removes its containing knowledge records, so no unlocalized process projection survives a localized authored-body-only view.

Run tools/build-process-diagram.cmd elevated. Real parser/SQLite capture and reopening, exact reviewed process history links, canonical HTML and full bundle validation accompany native FMX navigation and graph/step details. Tests cover reordered stable exception destinations, self-return, End, Unknown, escaped script-like labels, all 64 steps and rejected overflow, audience scope, draft protection and replaced/cleared knowledge. Regression suites cover existing Knowledge, publication, workflow and technical graphs plus the actual application/FMX build. Fixtures use the fixed ignored bin/process-diagram-validation folder.

## Maintain the business rule catalog

TDPHWorkspaceView.NewBusinessRule, ValidateBusinessRule and BusinessRuleBody define the existing-unit business_rule_definition contract: stable canonical rule_id GUID and nine bounded text fields. The human rule_code is required, 1 to 100 ASCII characters, with strict end matching; name is descriptive and at most 200 characters. Every text field is at most 4096 characters. Unknown remains an unresolved business value rather than inferred policy.

TDPHKnowledgeStore.SaveKnowledge validates project scope, business_rule kind, maintenance document, maintainer audience, title and canonical body. Identity and saved human rule code cannot change. Case-insensitive rule codes and GUIDs are unique within the selected project, including retired history. Existing revision guards, attributed review, immutable history and exact source/procedure links apply. No schema migration or helper unit was added.

BusinessRuleTab and BusinessRuleEditorTab are serialized FMX controls editable in the IDE. IsBusinessRule admits only typed rule definitions and original generic business_rule notes; typed process, workflow, data-quality and recovery records remain in their own registers. Generic notes keep their original attribution and body, with explicit Unstructured labeling. Structured conversion of an existing generic note is forbidden.

Run tools/build-business-rule.cmd elevated for real parser/SQLite, identifier collisions and immutability, malformed records, exact implementation/procedure links, named review/history, canonical publication and database reopening. Native normal and compact renders cover field editing, pending draft protection, filters, legacy notes and bounded paging. The fixed ignored bin/business-rule-validation fixture does not execute selected target code. Run tools/build-wp0.cmd for the actual application and FMX resource binding checks.

## Source and revision evidence

The following source files and resources define this guide. SHA256 values identify the inspected implementation. Rebuild the guides after validated changes and retain the associated acceptance evidence.

DPH.Analysis.Async.pas
`876a836f25b3d323ac4d6f635701fb4e8e97955fdff39c964368aa62f1a4c153`
DPH.Analysis.Explanations.pas
`8e4a430daf51b5f1d5c0e4e8a939862dd8b80123bd7333597b949ad054f85f06`

DPH.Analysis.Flow.pas
ee7610544411a64da399659f2af11e84408d285837a45205d610d2c37af8ef57
DPH.Analysis.Model.pas
93d7e969508ff4862a98f7e4d7b6cb690cf741792d5a0e4dbac9f012bc8c5035
DPH.Analysis.Persistence.pas
08609c3df54250a8f6712e22e991f7ad8d444352e7103ba0959d2feba4a2b3c7
DPH.Analysis.Quality.pas
50196d33a8d15cfeb52528dde93a2bfdc22b153e2533830b08e3ac097e14fd82
DPH.Analysis.SQL.pas
68ea587d47e2c7ebf660f85eed6b605f6076480ead3a461742b8290fec7a130e
DPH.Analysis.Symbols.pas
47c187a32ad1a040760bd9459e843d9c6fa7ced7b4ea5843280d2680127cc047
DPH.Knowledge.ActionPurpose.pas
bd548e1d5bbb886d3d3f2b5ba59d1cd86144a92877eba20596f956582740573f
DPH.Knowledge.ComponentDependency.pas
56bc1021fffa88f0a383a2a159b4c74328d735597f72c3ec279ef42ad2a2ccbe
DPH.Knowledge.Glossary.pas
9c50a06ccf405f786526e206e5e84ea956604eeb42e1a84b9f91c31cced70dcb
DPH.Knowledge.ProjectProfile.pas
e901a58b02c2aafe2cc21df4da7dced2807a9cc8840c325c72d9db64cedc44bd
DPH.Parse.Forms.pas
3cf7d49afd859e2d2749cb30e5fdd76120eaf05ef5223e8fb8bf6eb1a7fcd66c
DPH.Parse.Lexer.pas
6ce5bbc66f8170b17eaeda536b8e500ca9992131477c09bc8e1a33bb6dd1c7f0
DPH.Parse.Model.pas
bdecd758070044593a6a19bfd53b35f9a23286e2542194313222cdfee51d6557
DPH.Parse.Pascal.pas
016adbe9ba24b90d457455722782cdbbc657233bd1a07ba1bcdcd77f3340587e
DPH.Parse.Preprocessor.pas
34b7513019a41bc9911babfda72041addd563959ee54c54d0759b8cc9fa7fefc
DPH.Parse.Resources.pas
f63153edba8656b315956fdd8ccf52e45b1446d8192f9b84c96c153bf676258f
DPH.Project.Analysis.pas
5af3808df19a7c403b456342e45b9e2c8c63195368847b151f3c57ed461b5c2a
DPH.Project.Inventory.pas
061736891f0fab486996ff5d8132af1cf5e25a2f93a74fb3e03ad8fa04e2c266
DPH.Project.Source.pas
8efa444743f0e90831631df963f26a2542561efab63b26f1c5b41fff6b96994e
DPH.Publish.BrowserPolicy.pas
74d2f36c879f1b7b55133241c3e1abc28ce113c4d9f581a5123be83f94d705ce
DPH.Publish.Export.pas
ac49dff7e08016db2f8c720de3bac0749e52de82ccc4e7aa2b328f8032ceebe4
DPH.Publish.Model.pas
7068252c58364ae224e263ecc5fbf83bdb406943979f3ad752f4ca9f877de9d2
DPH.Publish.Translation.pas
978be4250a2332736f73cd4e56e85d5c2f0b0d3a6224a4db2f8f3b7b5f4cde20
DPH.Storage.Backup.pas
78e8c8de8af5120f8ce453793f947c5e2918e51dafc66f9e129c964fe28697bc
DPH.Storage.Database.pas
6d3fd07349ecba128a429784a068078eeb6a0c3a12caa5a73879c4af008fccc8
DPH.Storage.Types.pas
1e83ccbe6868979ff9d763a4f2bf4d64691c2b2343de6717391f03e204d5d219
DPH.UI.ActionPurpose.pas
6016874bdc24964ce2371431d77babc68f03a50f045e8d20e222e4989bcbf79d
DPH.UI.ComponentDependency.pas
dc711088afbccd032f4511b9fe3da95b89640b505edb1868ee7b47c7e2a64682
DPH.UI.DatabaseDictionary.pas
c55bc57ac98df6caa84f29c5b775b4915c8afaf3837859f3e197f8b1fffbf90b

DPH.UI.Glossary.pas
fb15295b29a9bbb9caa05cea723e737c81b2418a271f59fb6071dbffe37f8741
DPH.UI.Main.pas
8e6d9444643479eac6fe9f1902be8c40439cd61988806f28bf53da9f2a776e76
DPH.UI.ProjectProfile.pas
f3fe7ed94987e956999e7449887cd8ca259f009b3d4ac934c1e8e09b8e9fab59
DPH.Workspace.ActionPurposes.pas
e720b935f3d9f38f07348bead5f21c9c9e3ec89ad102f4dae0c6e902c0a101e4
DPH.Workspace.Catalog.pas
5fed34d1919dfd5b34216be36f92fc1764847a65d62112613b2a0e1d44a05940
DPH.Workspace.Commands.pas
9d737351356cf30b1f5bd6d2dee5c514bcafe16c6fcafe24be82b2f06c1a3d28
DPH.Workspace.Comparison.pas
d4953d401b70fbe65b8b9b9fe387431cb7e2c84afc5dab5c9170deafe9ceacce
DPH.Workspace.ComponentDependencies.pas
7d57eb7884d6e521414379a24ce9e64aa2816beca6de650b86828a16fc33af2b
DPH.Workspace.DatabaseConnections.pas
37fecd28b258661a57eb1ff955a2bf1f22757508b3925b98daedc0d6b789e1ce
DPH.Workspace.DatabaseSchema.pas
988548013d7465e7a95b4b1729c0fec5938f5d866afda932bfeab22639160bda
DPH.Workspace.DynamicControls.pas
abd5350eab05058b885b95e0c947a1badd54eba33877cbe9a448cac194db25ed
DPH.Workspace.FormInheritance.pas
b771383670720f9503454dc17f7d5e72ddd986a642fc526bc6aeeab6122129f8
DPH.Workspace.Graphs.pas
1d2985c6fa30e72a7ddb7c55f41a2a774e6e1517d5056507efb2666f18b47804
DPH.Workspace.Keyboard.pas
6e854d724e8360e3d5015d63077744636cf88f33eed5c9fcff175ef01e0f9923
DPH.Workspace.LayoutRules.pas
02dcd1a040fc209a557281fe8150bd40a59291a18a08fa48d5e07065dbac1c49
DPH.Workspace.Model.pas
b64efef8c916e772e0c39597eca056b5d892d4f3c7e68315d5b6250ad905643e
DPH.Workspace.Screen.pas
68305e6ec36664e76d05b8c9af21416dd51a32116d47ea55156977b4c10855de
DPH.Workspace.ScreenHistory.pas
bd05a76dcae7b4c872345021a56eab0b8b0108d954c1a5c35686321852082161
DPH.Workspace.Worker.pas
d6ac3ffff0c50dbcbb98f0813db1ea47769f0fa8e1f2e9418381c6735cc09953
DPH.UI.Main.fmx
38c763770038692796e73a1a0c0a793aa01db6bae96db716d4ec349b60550907
DPH.UI.ProjectProfile.fmx
62e7fef97972fdb29f1a878e70a99d8284b6f7d8cfbddb9f625bb535728c2142
DPH.UI.Glossary.fmx
3334c41ac643daa3596f1a5c9cb8fb65c3447daf77cf00006d6a0ca7a9d517d5
DPH.UI.ComponentDependency.fmx
3b604fcf033e37ebda4aa6968e1a9e6ffe11bf40416fac5fbdd39fa2599d7862
DPH.UI.ActionPurpose.fmx
9eb57da0310277432db54c9a80d8d03be09aa6163acd8895f04fa21f176f8e15
DPH.UI.DatabaseDictionary.fmx
5b40f12073d093c4e0b7f708440dd125662dc8e76a0b3c3bf30a4c749c69484e
DelphiProjectHandoff.dpr
2b922d5561c2ab3e2b65590583f646e29fc12c8b5c36a08fe401a6ebed90d3f9
DelphiProjectHandoff.dproj
fc131b87384bc80f6d391db9c015b7aaa41c098ff8b96d53abfcdb67ced18e8d